

第五章 Java 语言基础

理解类的基本结构、构造函数、类的属性、方法、静态成员（静态属性和静态方法）、main()方法、包、成员访问控制，类的封装。

5.1 类的基本结构

类和对象的区别

1，类是一个抽象的概念，它不存在于现实中的时间/空间里，类只是为所有的对象定义了抽象的属性与行为。就好像“Person（人）”这个类，它虽然可以包含很多个体，但它本身不存在于现实世界上。

2，对象是类的一个具体。它是一个实实在在存在的东西。

3，类是一个静态的概念，类本身不携带任何数据。当没有为类创建任何对象时，类本身不存在于内存空间中。

4，对象是一个动态的概念。每一个对象都存在着有别于其它对象的属于自己的独特的属性和行为。对象的属性可以随着它自己的行为而发生改变。

什么是类，什么是对象，类和对象之间的关系

类的概念：类是具有相同属性和行为的一组对象的集合。它为属于该类的所有对象提供了统一的抽象描述，其内部包括属性和行为两个主要部分。在面向对象的编程语言中，类是一个独立的程序单位，它应该有一个类名并包括属性说和行为两个主要部分。

属性：用来描述具体某个对象的特征的是属性，是静态的。比如：姚明身高 2.6 米多；小白的毛发是棕色的；二郎神额头上有只眼睛；

方法：每个对象有它们自己的行为或者是使用它们的方法，比如说一只狗会跑会叫等，我们把这些行为称之为方法，是动态的，可以使用这些方法来操作一个功能，

对象的概念：对象是系统中用来描述客观事物的一个实体，它是构成系统的一个基本单位。一个对象由一组属性和对这组属性进行操作的一组服务组成。从更抽象的角度来说，对象是问题域或实现域中某些事物的一个抽象，它反映该事物在系统中需要保存的信息和发挥的作用；它是一组属性和有权对这些属性进行操作的一组行为的封装体。

客观世界是由对象和对象之间的联系组成的。类与对象的关系就如模具和铸件的关系

类的实例化结果就是对象，而对一类对象的抽象就是类。类描述了一组有相同特性（属性）和相同行为（方法）的对象。

new 关键字的作用就是在你声明了一个对象后，给对象分配相应内存。强类型，较高效。能调用任何 public 构造。1、创建对象，实例化对象；2、实例化对象，赋予对象空间，即堆内存地址；3、调用构造函数；

比如当我们创建一个对象：Student stu= new Student（）；在这里的 new 具体作用为：1、首先要明确 stu 是父类的一个引用，没有实际在堆中分配空间。2、其次 new Student（）的作用了：new 是在堆中为对象 stu 申请了一块空间。其中 new 也实际上是在调用了父类的构造方法。

基本结构如下

<访问权限控制>class 类名 { }

```
// 类的基本结构
// 由class关键字和类名构成
public class Teacher {
    // 类中的代码
}
```

5.2 类的构造函数

构造函数的特点：

- 1、跟类的名称相同
- 2、必须为 public 修饰
- 3、返回值类型必须 void，一般情况下不写。
- 4、构造函数可以重载。

举例：

```
public class Teacher {
    // 类中的代码
    // 构造函数
    public Teacher(){
    }
}
```

5.3 类的成员

1、属性

属性也叫类的成员变量，例如：

```
public class Teacher {
    // 属性
    private String nameString; // 姓名属性
    private int age; // 年龄属性
}
```

2、方法

表示这个类的行为特征。

```
public class Teacher {
    // 属性
    private String nameString; // 姓名属性
    private int age; // 年龄属性

    // 构造函数
    public Teacher(){

    }

    // 类的方法
    public void teach(){
        System.out.println("I can teach!");
    }
}
```

```
}  
}
```

常量

常量的修饰符为 `final`，静态常量的语法声明如下：

```
Public static final int PI=3.1425926;
```

5.4 类的静态成员

使用 `static` 关键字把成员声明为静态成员。分为静态属性和静态方法，成为类的变量和方法。`Static` 表示属于类，不必创建对象就可以使用；

举例：

```
public class JavaMain {  
    public static void main(String[] args) {  
        Teacher teacher=new Teacher();  
        // 通过对象来调用普通方法  
        teacher.teachMath();  
        // 通过类来调用静态方法  
        Teacher.teachEnglish();  
        // 通过类来调用惊天成员（属性）  
        String string=Teacher.nameString;  
    }  
}  
  
public class Teacher {  
    // 属性  
    public static String nameString="我是一个静态变量";// 姓名属性  
    private int age;    // 年龄属性  
    // 构造函数  
    public Teacher(){  
    }  
    // 类的（普通）方法  
    public void teachMath(){  
        System.out.println("I can teach Math!");  
    }  
    // 静态方法  
    public static void teachEnglish(){  
        System.out.println("I can teach English!");  
    }  
}
```

静态的方法有个很重要的特点：在静态的方法中，无法访问普通成员（成员和方法），但能够访问静态的，举例：（下面的方法引用属性会报错）。非静态的方法能够访问静态和非静态的成员。

```

11
12 // 类的（普通）方法
13 public void teachMath(){
14     this.age=15;
15     this.nameString="Mr Huang";
16     teachEnglish();
17     System.out.println("I can teach Math!");
18 }
19 // 静态方法
20 public static void teachEnglish(){
21     // 静态的方法不能访问非静态的属性和方法
22     age=15;
23     teachMath();
24     // 静态方法能访问静态的属性和静态的方法
25     teachChinese();
26     nameString="Mr Huang";
27     System.out.println("I can teach English!");
28 }
29
30 public static void teachChinese(){
31     System.out.println("I can teach English!");
32 }
33 }

```

5.5 main()方法

在 Java 中，main()是 java 应用程序的入口方法。它的主要特点有：

- 1、名称必须为 main
- 2、必须为 public
- 3、返回值必须为 void
- 4、必须声明为静态的

```

public static void main( String[] args ){
}

```

5.6 包

作用：方便组织管理。

如何定义一个包：语法格式 **package <包名>**

导入包： **ctrl+shift+O**

语法格式 1: **import <包名>.***

语法格式 2: **import <包名>.类名**

有如下特点：

- 1、在 package com.huang.libs 之前，不能出现任何代码。
- 2、同名包问题。

遇到同名包导入，又两者都必须要用到的情况下，请做如下处理：

```

import java.sql.*;
import java.util.*;
public class JavaMain {
    public static void main(String[] args) {
        // 用完整的包名+类名来创建对象
        java.sql.Date date=new java.sql.Date(20160630);
        java.util.Date date2=new java.util.Date();
    }
}

```

}

5.7 成员访问控制

成员访问控制有 4 种类型：

- 1、公共类型 `public`
- 2、私有类型 `private`
- 3、默认类型 `default`
- 4、保护类型 `protected`

`public` 特点是什么？

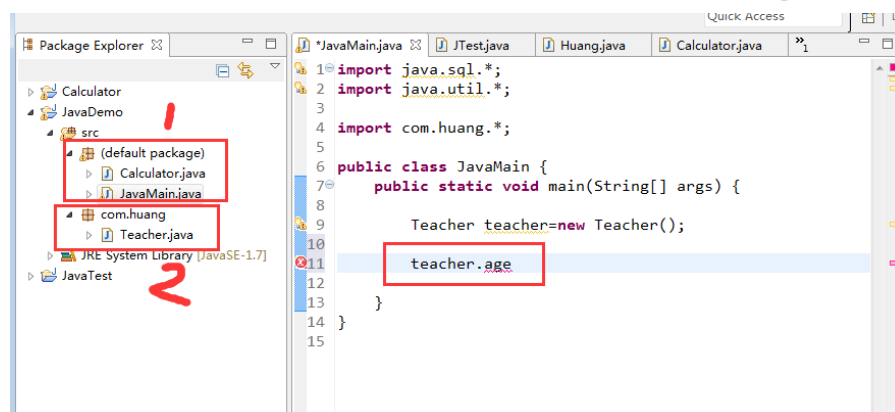
1、当一个成员被声明为 `public` 时，所有其他类（无论该类属于哪个包），均能够访问该成员。

`private` 有什么特点？

当一个成员被声明为 `private` 时，其他类（本类除外）都不能访问该成员。

`default` 有什么特点？

当一个成员前面，没有写任何一个修饰符，其访问权限为默认类型 `default`。以包为界限，同一个包内，该成员可以看成是一个 `Public` 类型；这个包之外，被视为 `private` 类型。



`protected` 有什么特点呢？

成员被 `protected` 修饰，规则跟默认类型（`default`）几乎一样。当访问该成员的类型位于同一个包内，则该类型相当于 `public` 类型，反之，则为 `private`。

参考文献：<https://blog.csdn.net/guidgeek/article/details/112558077>

	同一类	同包		不同包	
		子类	非子类	子类	非子类
Private	是	否	否	否	否
友好型	是	是	是	否	否
Protected	是	是	是	是	否
Public	是	是	是	是	是

<https://blog.csdn.net/guidgeek>

- 1、 子类可以继承 public 和 protected 类型的属性和方法。
- 2、 父类中的 private 到子类中不可见。

示例代码：

//同一个包中：类的继承与访问控制

```
public class JavaDemo {
    public static void main(String[] args) {
        Manager m=new Manager();
        m.id=123456; // 继承父类的公有属性
        m.age=35;    // 继承父类的保护属性
        // m.salary=4000; // 错误，父类私有成员到子类中不可见
    }
}
```

//子类的定义-> 雇员

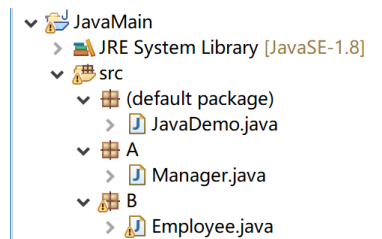
```
class Employee{
    // 属性
    public int id;      // 公有类型属性
    protected int age; // 保护类型的属性
    private int salary; // 私有类型的属性
    // 方法
    public int getID() { // 共有类型的方法
        return id;
    }
    protected int getAge() { // 保护类型的方法
        return age;
    }
    private int getSalary() { // 私有类型的方法
        return salary;
    }
}
```

//子类的定义-> 经理

```
class Manager extends Employee{
```

}

测试不同包中的情况：



5.8 类的封装

封装是面向对象的术语，其含义很简单，就是把东西包装起来。

一般来说：

- 1、属性需要包装，用 `private` 修饰（相当于包装起来）。
- 2、方法属于行为特征，用 `public` 修饰，无需包装。

特殊情况下：

属性也需要 `public` 修饰，不能包装。

有些很特殊的方法，是需要包装的，用 `private` 修饰

综上所述：包装是根据你的需要。

对于封装过的属性，可以通过下面的方法来维护其使用。

通过 `getX()` 来访问该包装的成员变量（属性）。

通过 `setX()` 来修改被包装的成员变量（属性）。

5.9 抽象类

在 Java 语言中，用 `abstract` 关键字来修饰一个类，这个类就叫抽象类。抽象类至少包含一个或者多个抽象方法。抽象类可以看作对类的进一步抽象，在面向对象领域，抽象类主要用来进行类型隐藏。语法定义如下：

```
public abstract class 类名{  
    // 属性  
    // 方法的定义  
}
```

抽象类中含有抽象方法，抽象方法需要用 `abstract` 修饰，是没有函数体的，如果写了方法体也就是包含大括号，编译器会报错。

具体语法如下：

```
public abstract void 函数名(int x,int y);
```

下面通过一个例子，来综合介绍抽象类和抽象方法的应用。

```
public class JavaMain {  
    public static void main(String[] args) {  
        // 用抽象类来创建一个应用，并实例化;  
        Shape shape=null;  
        // shape=new Shape(); // 错误  
        shape=new Circle(4);  
        Circle circle=new Circle(3);// 正确  
        circle.drawDot(0, 0);  
    }  
}
```

```

// 抽象类
abstract class Shape{
    private double PI=3.14159;
    // 至少含有一个抽象函数
    public abstract void drawDot(int x,int y);

    // 可以包含普通函数
    public void drawLine(int x1,int y1,int x2,int y2) {
        System.out.println("调用了画线函数! ");
    }
}

// 写抽象类是一定要写子类的，否则没有意义
class Circle extends Shape{
    // 属性
    private double radis=0;
    // 构造方法
    public Circle(double r) {
        radis=r;
    }
    @Override
    public void drawDot(int x, int y) {
        System.out.println("需要对抽象方法重写（实现）！");
    }
}

```

5.10 接口

接口是抽象类功能的另一种实现方法，可将其想象为一个“纯”的抽象类，因为接口中所有的方法都是抽象方法，都没有方法体。因此，可以把接口看成是特殊的抽象类。接口与抽象类都是定义多个类的共同特征。Java 允许一个类实现多个接口，从而实现了比多重继承更加强大的能力，并且具备更加清晰的结构。

接口的语法定义如下：

```

[接口修饰符] interface 接口名称 [extends 父类名] {
    // 静态常量
    // 方法原型
}

```

接口与一般类一样，本身具有数据成员和方法成员，但是要求数据成员要赋初值，且不能被更改。方法成员必须是抽象的，即方法体必须为空，不能有方法体。

接口定义举例：

```

public class JavaMain {
    public static void main(String[] args) {
        //CharStorage c=new CharStorage();
        CharStorage c=new MyChar(); // 正确
        c.put('A');
    }
}

interface CharStorage{
    void put(char c);
    // char get() {}; // 错误，不能有大括号
}

class MyChar implements CharStorage{

```



```

@Override
public void put(char c) {
    System.out.println("对抽象方法的具体实现!");
}
}

```

应用举例 2:

```

public class JavaMain {
    public static void main(String[] args) {
        // Shape2D shape2d=new Shape2D(); // 错误, 接口不能实例化对象
        Shape2D shape2d=new Circle(3); // 正确, 可以创建一个引用, 然后用
子类去实例化
        System.out.println(shape2d.area());

```

```

        Circle circle=new Circle(4);
        System.out.println("Circle area =" + circle.area());
        Rectangle rect=new Rectangle(3, 4);
        System.out.println("Rect area =" + rect.area());

    }
}

```

```

interface Shape2D{
    // double PI; // 错误, 定义的属性要初始化
    final double PI=3.14; // 正确, final 可以省略
    public abstract double area(); // 抽象方法, 不需要定义处理方式
}

```

// 下面定义 Circle 实现 Shape2D 接口

```

class Circle implements Shape2D{
    double radius;
    public Circle(double r) { // 构造方法
        radius=r;
    }
    @Override
    public double area() {
        return PI*radius*radius;
    }
}

```

// 定义 Rectangle 实现 Shape2D 接口

```

class Rectangle implements Shape2D{
    // 属性定义
    int width,height;
    // 方法定义

```

```

public Rectangle(int w,int h) {
    width=w;
    height=h;
}

@Override
public double area() {
    return width*height;
}
}

```

5.11 内部类

在 Java 中，可以将一个类定义在另一个类里面或者一个方法里面，这样的类称为内部类。广泛意义上的内部类一般来说包括这四种：成员内部类、局部内部类、匿名内部类和静态内部类。下面就先来了解一下这四种内部类的用法。

1、成员内部类

成员内部类是最普通的内部类，它的定义为位于另一个类的内部，形如下面的形式：

```

class Circle {
    private double radius = 0;
    private int x=0, y=0;
    public Circle(double radius) {
        this.radius = radius;
    }
    class Point { //内部类
        int x=10,y=10;
        public Point(int xx,int yy) {
            this.x=xx;
            this.y=yy;
        }
        public int getJudge() {
            double dist=(this.x-x)*(this.x-x)+(this.y-y)*(this.y-y);
            if (dist<radius*radius) {
                System.out.println("点在圆的内部! ");
                return 1;
            }
            else if (dist>radius*radius) {
                System.out.println("点在圆的外部! ");
                return -1;
            }
            else {
                System.out.println("点在圆上! ");
                return 0;
            }
        }
    }
}
}

```

结论：

这样看起来，类 Piont 像是类 Circle 的一个成员，Circle 称为外部类。内部类 Point 成员可以无条件访问外部类的所有成员属性和成员方法（包括 private 成员和静态成员）。

2、局部内部类

局部内部类是定义在一个方法或者一个作用域里面的类，它和成员内部类的区别在于

局部内部类的访问仅限于方法内或者该作用域内。

```
class Shape {
    public Shape() {
    }
}
class Circle{
    public Circle() {
    }

    public Shape getArea() {
        // 在方法里面，定义局部内部类
        class Rect extends Shape{
        }
        Rect rect=new Rect();
        return rect;
    }
}
```

注意，局部内部类就像是方法里面的一个局部变量一样，是不能有 `public`、`protected`、`private` 以及 `static` 修饰符的。

3、 匿名内部类

匿名内部类应该是平时我们编写代码时用得最多的，在编写事件监听的代码时使用匿名内部类不但方便，而且使代码更加容易维护。下面这段代码是一段 Android 事件监听代码：

```
class Shape {
    public Shape(){
        JButton button=new JButton();
        button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                System.out.println("按钮侦听事件");
            }
        });
    }
}
```

匿名内部类是唯一一种没有构造器的类。正因为其没有构造器，所以匿名内部类的使用范围非常有限，大部分匿名内部类用于接口回调。匿名内部类在编译的时候由系统自动起名为 `Outter`

4、 静态内部类

静态内部类也是定义在另一个类里面的类，只不过在类的前面多了一个关键字 `static`。静态内部类是不需要依赖于外部类的，这点和类的静态成员属性有点类似，并且它不能使用外部类的非 `static` 成员变量或者方法，这点很好理解，因为在没有外部类的对象的情况下，可以创建静态内部类的对象，如果允许访问外部类的非 `static` 成员就会产生矛盾，因为外部类的非 `static` 成员必须依附于具体的对象。

```
public class JavaMain {
    public static void main(String[] args) {
        Outter.Inner inner = new Outter.Inner();
    }
}
class Outter {
```

```

        public Outter() {
        }
        static class Inner {
            public Inner() {
            }
        }
    }
}

```

5.12 常用类---String 类

字符串广泛应用 在 Java 编程中，在 Java 中字符串属于对象，Java 提供了 String 类来创建和操作字符串。

String 类在 java.lang 包中，java 使用 String 类创建一个字符串变量，字符串变量属于对象。java 把 String 类声明的 final 类，不能有子类。String 类对象创建后不能修改，由 0 或多个字符组成，包含在一对双引号之间。

(1) String 类的使用

```

// 1 创建一个 String 对象
String string="Anhui University";
String string2=new String("Hello World !");
// 2 获取字符的长度
int len=string.length();
System.out.println(len);
// 3 查找
char ch=string.charAt(0);
int index=string.indexOf("A");
boolean bool=string.contains("Anhui");
boolean isEn=string.isEmpty();
System.out.println("ch="+ch+",index="+index+",
bool="+bool+",isEn"+isEn);
// 4 提取字符串中的子串
String substr=string.substring(0,5);
System.out.println(substr);
// 5 比较两个字符串
String str1="abc";
String str2="ABC";
int a= str1.compareTo(str2);
int b= str1.compareToIgnoreCase(str2);
boolean c=str1.equals(str2);
boolean d=str1.equalsIgnoreCase(str2);
System.out.println("a="+a+",b="+b+",c="+c+",d="+d);
// 6 连接、合并
String string3= "Hello".concat(" world").concat("!");
String sss="a="+a+",b="+b+",c="+c+",d="+d;
String string4=String.format("整数%d"+",整数%d"+",布尔%s" ,a,b,c);
System.out.println(string4);

```

5.13 常用类---StringBuffer 类

String 和 StringBuffer 都可以存储和操作字符串，即包含多个字符的字符串数据。String 类是字符串常量，是不可更改的常量。而 StringBuffer 是字符串变量，它的对象是可以扩充和修改的。

```
// 1 创建一个 StringBuffer 对象
StringBuffer strbuf0=new StringBuffer();
StringBuffer strbuf1=new StringBuffer("Anhui University");

// 2 获取字符的长度
int len=strbuf0.length();
System.out.println(len);

// 3 添加
strbuf1.append(" is ").append(211).append(" very");
strbuf1.insert(strbuf1.length()," nice !");
System.out.println(strbuf1);

// 4 替换
strbuf1.replace(strbuf1.length()-6, strbuf1.length(), "OK !");
System.out.println(strbuf1);

// 5 子串
System.out.println(strbuf1.substring(strbuf1.length()-4));

// 6 删除和清空
strbuf0=strbuf1.delete(0, 5);
System.out.println(strbuf0);
strbuf0=new StringBuffer(); // 清空，原来的会编程垃圾
strbuf1.delete(0, strbuf1.length()); // 推荐的情况方法，因为能清空
```

缓存

```
// 7 StringBuffer 转成 String
StringBuffer sb1=new StringBuffer("abc");
String string=new String(sb1); // 通过构造函数的方法转变
System.out.println(string);
String string2=sb1.toString();// 通过 StringBuffer 的 toString()方法
System.out.println(string2);
```

法

5.14 编程练习

1、 首先，我们在工程中新建一个类，类名称为Animal，他的属性和方法定义如下：

```
public class Animal {
    // 动物的属性
    private int legs;// 腿的数目
    private int age; // 寿命
    public boolean bIfHasMao;// 是否有毛
```

```

private String name;// 名字
// 定义动物的眼睛
// private Eye eye;
// 构造函数
public Animal(int leg,int age,String name){
    // 用于初始化对象
    this.legs=leg;
    this.age=age;
    this.name=name;
    this.bIfHasMao=false;
    this.age=0;
}
// 定义动物的行为
// 设置动物眼睛数目
/*public void setEyes(int n){
    eye.setEyes(n);
    System.out.println("已经设置好动物的眼睛数目: "+n);
}*/
// 获取动物的眼睛数目
/*public Eye getEyes(){
    return eye;
}
*/
// 设置动物的名字
public void setName(String name){
    this.name=name;
    System.out.println("已经设置好动物的名字: "+name);
}

// 获取动物的名字
public String getName(){
    return name;
}
// 动物能叫
public void canBite() {
    System.out.println("Hello,becafull, I can Bite!");
}
}
}

```

2、现在我们来使用上面创建的类，在主类中编写代码如下：

```

public class JavaMain {
    public static void main(String[] args) {
        // 用自己写好的类，来创建一个对象，并调用构造函数初始化
        Animal animal=new Animal(4, 1, "xiaoqiang");
        // 给动物换一个名字
        animal.setName("Jack");
        // 非常重要，引用嵌套
        //animal.getEyes().setEyes(3);
        String animalName=animal.getName();
        System.out.println(animalName);
    }
}

```

如何编写一个Math类？

该类是仿照Jre系统中的数学计算类Math而写的，其实类库中的类，我们也能写出来，这个例子，我们就要是仿照类库中的功能，自己编写一个Math类，来完成数学计算中的求最大值和最小值，绝对值等。

1、首先我们来编写一个Math类

```

public class Math {
    private double PI;
    private double E;

    // 构造函数
    public Math(){
    }
}

```

```

// 求最大值
public double getMax(double d1, double d2){
    double max=0;
    // 比较过程得到最大值
    if (d1>d2) {
        max=d1;
    }else{
        max=d2;
    }
    // 返回最大值
    return max;
}

// 求最小值
public double getMin(double d1, double d2) {
    double min = 0;
    // 比较过程得到最大值
    if (d1 > d2) {
        min = d2;
    } else {
        min = d1;
    }
    // 返回最大值
    return min;
}

// 求绝对值
public double getAbs(double d){
    double abs;
    if (d<0) {
        abs=-d;
    }else{
        abs=d;
    }
    return abs;
}
}

```

2、然后呢，我们在外界来调用它，希望Math类能为我们做点事情。

```

public class JavaMain {
    public static void main(String[] args) {
        // 对于Math类来说，这两个数是外界的数据
        // 我们希望用Math类来为外界做点事情，比如求最大值和最小值等
        double d1 = 3.255555;
        double d2 = -444554;
        // 创建一个Math对象，
        Math math = new Math();
        // 用 Math对象，来为外界求解最大值，把结果输出
        double max = math.getMax(d1, d2);
        System.out.println("两个数中的最大值是： " + max);
        // 用 Math对象，来为外界求解最小值，把结果输出
        double min = math.getMin(d1, d2);
        System.out.println("两个数中的最小值是： " + min);
        // 用 Math对象，来为外界求解d1的绝对值，把结果输出
        double d1_abs = math.getAbs(d1);
    }
}

```

```

        System.out.println("d1的绝对值是: " + d1_abs);
        // 用 Math对象, 来为外界求解d1的绝对值, 把结果输出
        double d2_abs = math.getAbs(d2);
        System.out.println("d2的绝对值是: " + d2_abs);
    }
}

```

如何编写一个计算器类?

用计算器类来帮主程序实现加减乘除的功能。在计算器类Caculator中, 编写代码如下:

```

public class Calculator {

    // 构造函数
    public Calculator(){
        System.out.println("你成功创建了一个计算器! ");
    }

    // 实现加法功能
    public double add(double d1,double d2){
        double sum=d1+d2;
        return sum;
    }

    // 实现减法功能
    public double sub(double d1,double d2){
        double subtract=d1-d2;
        return subtract;
    }

    // 实现乘法功能
    public double multiply(double d1, double d2){
        double mul=d1*d2;
        return mul;
    }

    // 实现除法功能
    public double div(double d1,double d2) {
        double diva=d1/d2;
        return diva;
    }
}

```